

[University / Institution Name]

[Department of Computer Science and Engineering]

MatrixCanvas

An Interactive Tool for Visualising Linear Algebra Transformations

Software Design Report

Class Diagram, Software Design Architecture Diagram & SRS Appendix

Submitted by

Name	Student ID
Faiyaz Tahmid	230042107
Abrar Faiyaz	230042108
Ittesaf Ithun	230042125
Tamim Naser	230042133

Course: [Course Code – Course Title]
Instructor: [Instructor Name]
Submission Date: 26 July 2026

Contents

1	Introduction	2
2	Object-Oriented Design Overview	2
3	Class Diagram	2
3.1	Explanation of the Class Diagram	3
4	Software Design Architecture Diagram	5
4.1	Explanation of the Architecture Diagram	5
5	Conclusion	6
A	Software Requirements Specification	7

1 Introduction

MatrixCanvas is a web application built to help students build geometric intuition for 2×2 linear transformations. A user edits the four entries of a transformation matrix and immediately sees its effect on the standard basis vectors, the coordinate grid, and any custom vectors they add, animated smoothly between the identity transform and the target transform. Alongside the visual, a properties panel reports the determinant, trace, rank, eigenvalues, and qualitative properties (invertible, orthogonal, symmetric) of the current matrix.

The application is implemented as a single-page front-end using the following technology stack:

- **React 18** with **TypeScript** – component-based UI, strong static typing of the domain model.
- **Zustand** – a minimal, hook-based global state container.
- **HTML5 Canvas API** – immediate-mode rendering of the transformed grid and vectors.
- **GSAP** – tweening engine driving the identity $\rightarrow$ target animation.
- **React Router** – client-side routing between the *Playground* and *Learn* pages.
- **Vite & Tailwind CSS** – build tooling and utility-first styling.

This report documents the *software design* of MatrixCanvas: the object model that underlies the application (Section 3) and the layered architecture that organises the code base (Section 4). The team’s previously submitted Software Requirements Specification (SRS) is attached as Appendix A.

2 Object-Oriented Design Overview

MatrixCanvas follows an **object-oriented approach** at its core: the mathematics of the application is encapsulated in a dedicated **Matrix2x2** class that owns its data and exposes behaviour (determinant, trace, rank, eigenvalues, inversion, interpolation) as methods, rather than scattering these computations across the UI. The application’s state is likewise modelled as a strongly-typed object (**AppStore**) composed of well-defined data interfaces (**CustomVector**, **Preset**). The presentation layer (React components) is kept deliberately thin: components read from and write to this object model but do not themselves hold business logic.

Because of this separation, the design is documented with the two diagrams requested for OO projects:

1. A **Class Diagram** (Section 3) of the core domain and state objects, their attributes, methods, and relationships.
2. A **three-layer Software Design Architecture Diagram** (Section 4) showing how the Presentation, State-Management, and Domain layers are organised and how they communicate.

3 Class Diagram

Figure 1 shows the core object model of MatrixCanvas: the **Matrix2x2** domain class, the **AppStore** application state, and the two plain data interfaces it depends on, **CustomVector** and **Preset**. UI components (documented in Section 4) consume this model through the **useAppStore** hook and are intentionally left out of this diagram so that the true object model stands out clearly.

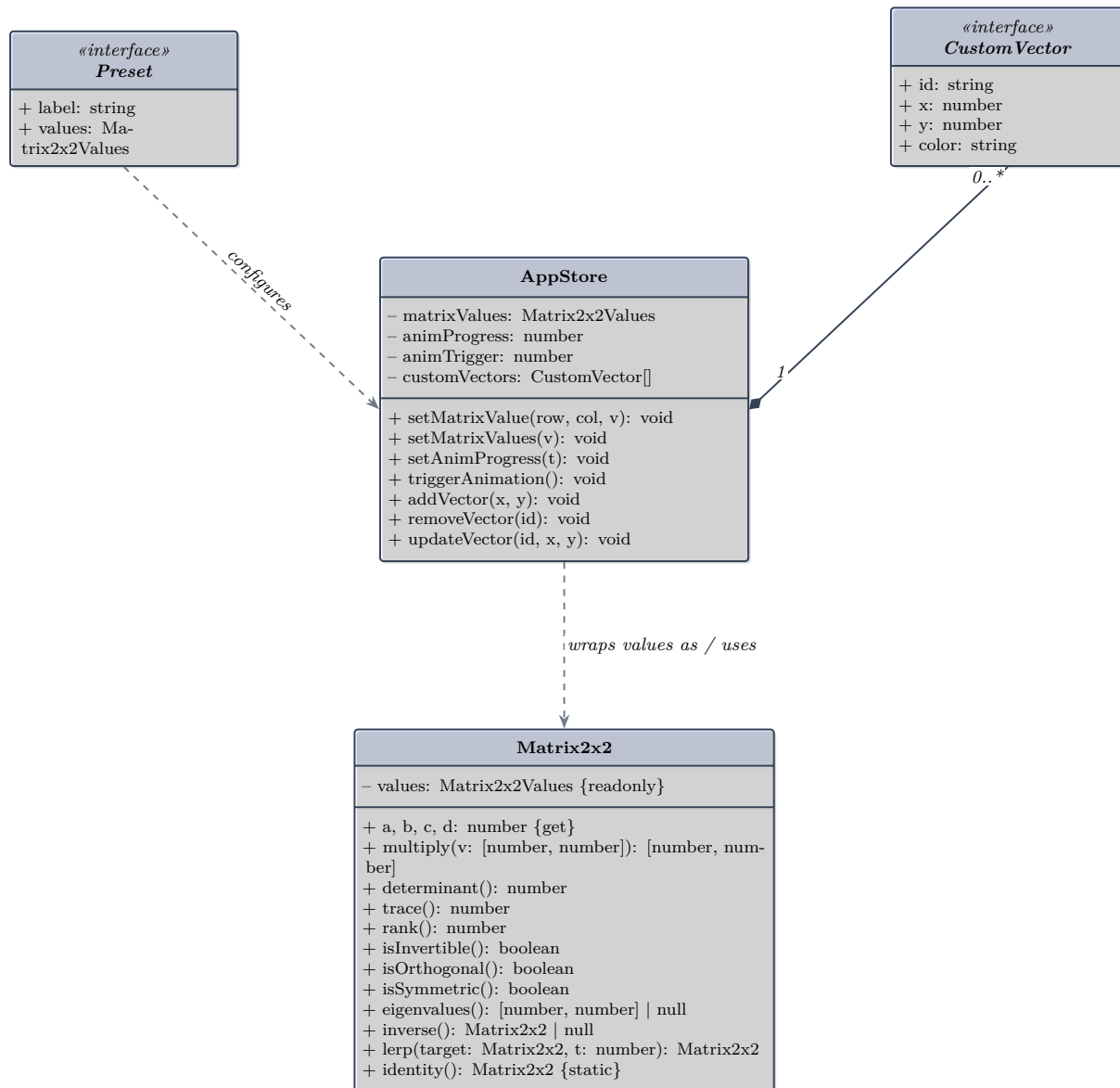


Figure 1: Class diagram of the MatrixCanvas core object model. Solid diamond = aggregation, dashed arrow = dependency (“uses”).

3.1 Explanation of the Class Diagram

Matrix2x2

is the domain class that encapsulates all linear algebra behaviour. It stores its four entries in a read-only **Matrix2x2Values** tuple and exposes accessors (**a**, **b**, **c**, **d**) alongside computed properties – determinant, trace, rank, invertibility, orthogonality, symmetry, and (real) eigenvalues – plus two operational methods: **multiply** (applies the matrix to a 2-vector) and **lerp** (linearly interpolates between two matrices, which drives the identity-to-target animation). **inverse** and the static **identity** factory return new **Matrix2x2** instances, keeping the class immutable.

AppStore

is the single source of truth for the application, implemented with Zustand. It holds the raw matrix values, the animation progress/trigger counters, and the list of user-added vectors, and exposes action methods that are the *only* way this state may be mutated. Components

never mutate state directly – they call an action, and the store notifies all subscribers.

CustomVector

is a plain data interface describing one user-added vector (position and display colour). **AppStore** aggregates zero or more of these ($1 - 0..*$); a vector's lifetime is entirely owned by the store.

Preset

is a plain data interface pairing a human-readable label with a canonical **Matrix2x2Values** tuple (e.g. “Rotate 45°”, “Horizontal Shear”). Selecting a preset configures **AppStore** by calling **setMatrixValues**.

Dependency AppStore $\dashrightarrow$ Matrix2x2

reflects that the store keeps only the *raw* matrix values; whenever a component needs derived mathematical properties or must apply the transform, it wraps the current **matrixValues** in a fresh **Matrix2x2** instance (e.g. `new Matrix2x2(matrixValues)` in **PropertiesPanel**, **VectorPanel**, and **TransformCanvas**). This keeps the store JSON-serialisable and the class purely computational.

4 Software Design Architecture Diagram

MatrixCanvas follows a **three-layer architecture**: a *Presentation Layer* of React components, a *State-Management Layer* holding the single Zustand store, and a *Domain / Business-Logic Layer* containing the mathematics. Figure 2 shows the three layers and the direction of communication between them.

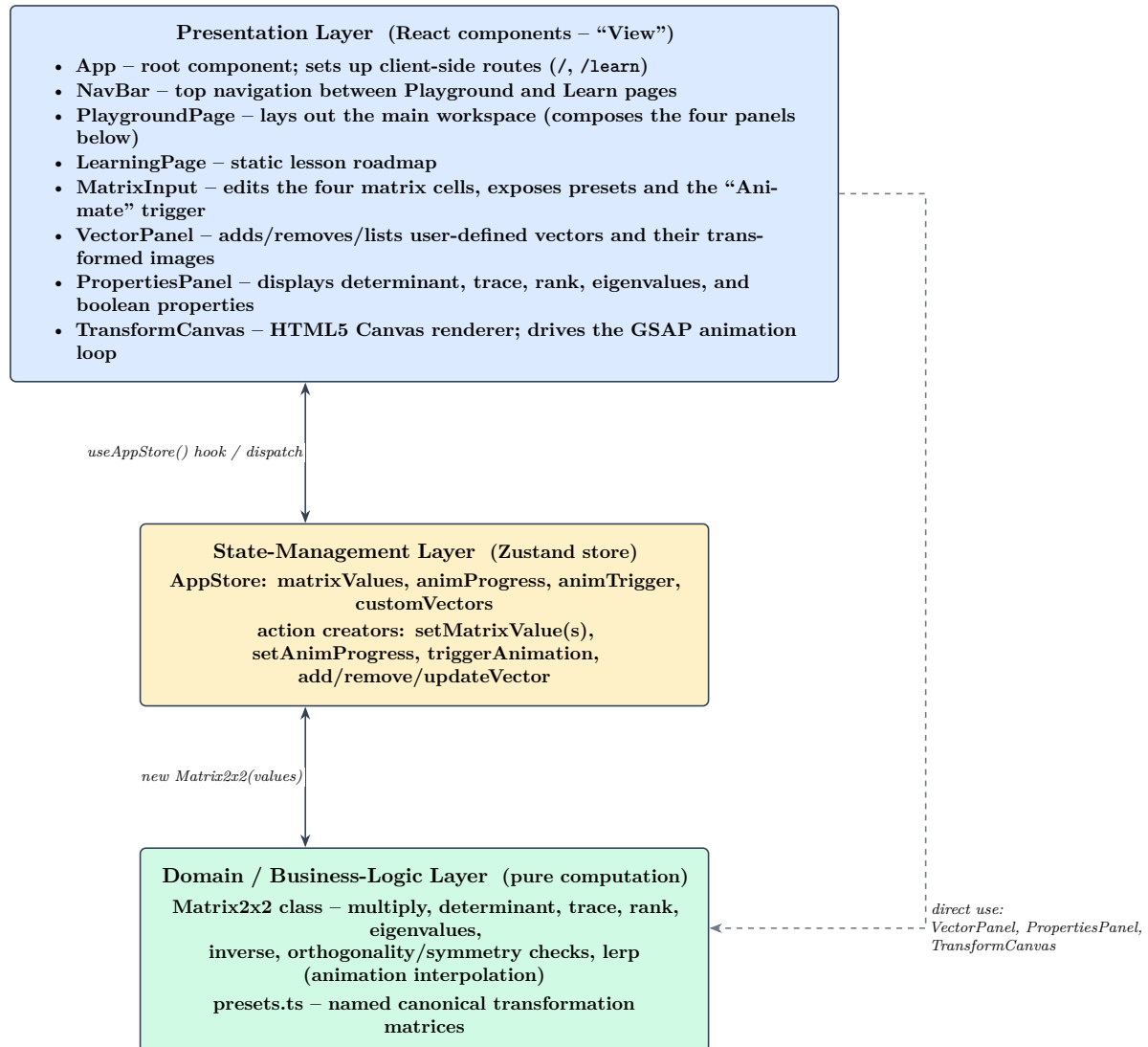


Figure 2: Three-layer software design architecture of MatrixCanvas. Solid double-headed arrows show the normal read/write flow through the state layer; the dashed arrow shows components instantiating `Matrix2x2` directly for derived computations.

4.1 Explanation of the Architecture Diagram

Presentation Layer

contains every React component. Its only responsibilities are rendering and capturing user input (typing matrix values, adding a vector, clicking “Animate”). It never computes a determinant or an eigenvalue itself.

State-Management Layer

is the single `AppStore` created with Zustand. Every component that needs live data calls the `useAppStore` hook (a selector subscription), and every user action is expressed as a call

to one of the store’s action methods – this keeps all state transitions in one auditable place. **TransformCanvas** additionally uses `useAppStore.subscribe` to mark its canvas “dirty” and redraw on the next animation frame, instead of re-rendering the whole React tree on every tick.

Domain / Business-Logic Layer

contains **Matrix2x2** and **presets.ts**. This layer has no knowledge of React or Zustand: it is plain, unit-testable TypeScript that could be reused in a completely different UI.

Layer communication

is intentionally not a strict pass-through: while **MatrixInput** only ever touches the State layer (it edits raw numbers), **VectorPanel**, **PropertiesPanel** and **TransformCanvas** read the raw **matrixValues** out of the store and then construct a **Matrix2x2** locally to obtain derived values or apply the transform. This shortcut (dashed arrow) avoids storing derived, recomputable data in global state.

5 Conclusion

The class diagram and the three-layer architecture diagram together describe how MatrixCanvas is designed: a small, immutable **Matrix2x2** domain class carries all linear-algebra behaviour; a single Zustand **AppStore** mediates all state changes; and a purely presentational React component tree renders the result and forwards user intent back to the store. This separation keeps the mathematics testable in isolation from the UI, and keeps the UI free of business logic. The team’s full functional and non-functional requirements are documented in the attached SRS (Appendix [A](#)).

A Software Requirements Specification

The Software Requirements Specification (SRS) previously submitted for this project is attached below.

*Instructions for the team: export your SRS document to a PDF named **srs.pdf**, place it in this same folder, and uncomment the line below (requires the **pdfpages** package, already loaded at the top of this file).*